

Docker

Réalisé par :

Aymane ALAHYANE

Yassine EL AOUNI

Encadré par :

Prof. Charaf OUADDI

Plan

- 1 Introduction et Contexte
- 2 Les Machines Virtuelles (VMs)
- 3 La Révolution Docker
- 4 Concepts Clés de Docker
- 5 Conclusion
- 6 Références

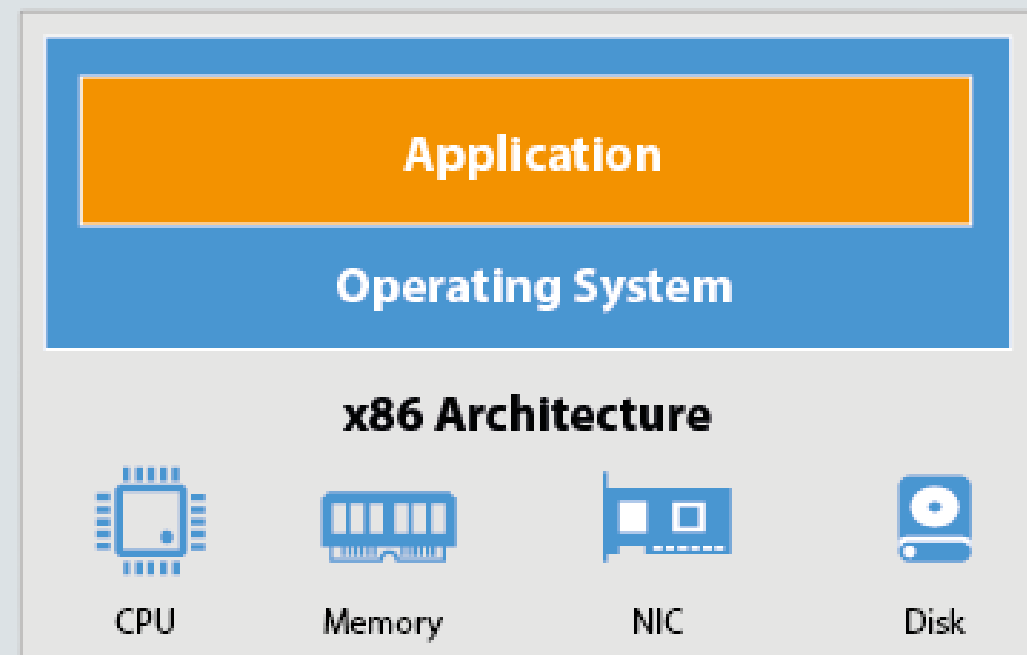
Contexte

- Le développement logiciel moderne est devenu plus complexe : les applications dépendent de nombreuses bibliothèques, services, et doivent fonctionner sur différents environnements.
- Cette complexité a mis en évidence des problèmes récurrents :
 - Des erreurs dues à des différences d'environnement.
 - Des déploiements difficiles à reproduire.
 - Des conflits entre dépendances de projets.
- Pour résoudre ces problèmes, plusieurs solutions ont été proposées au fil du temps : machines virtuelles, environnements virtuels, etc.
- C'est dans ce contexte qu'est né Docker, un outil qui a changé la façon dont les développeurs construisent, livrent et exécutent des applications.

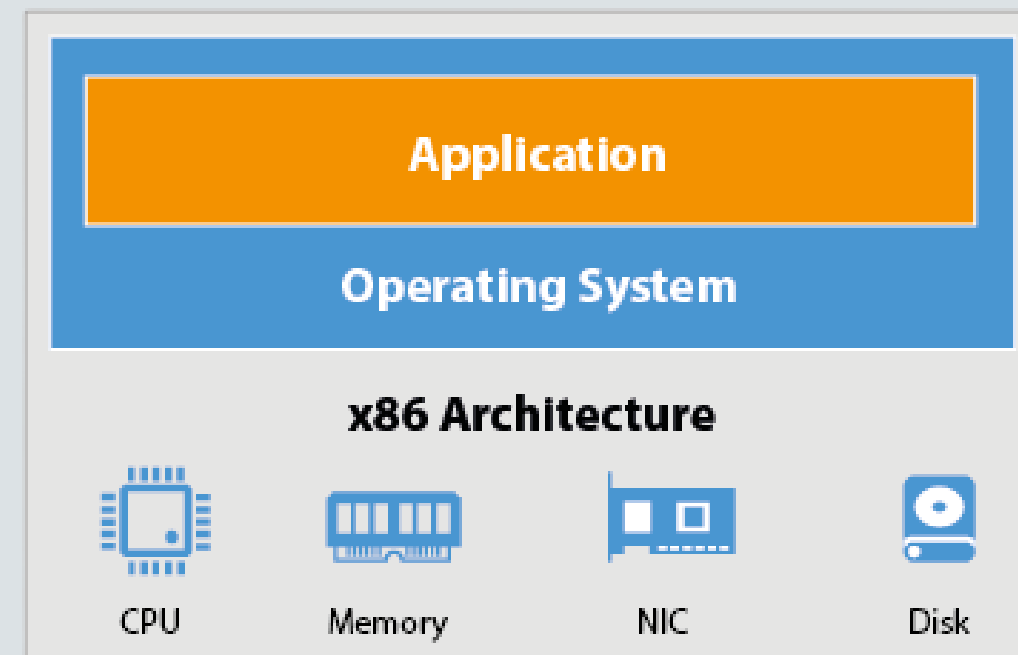


L'infrastructure traditionnelle

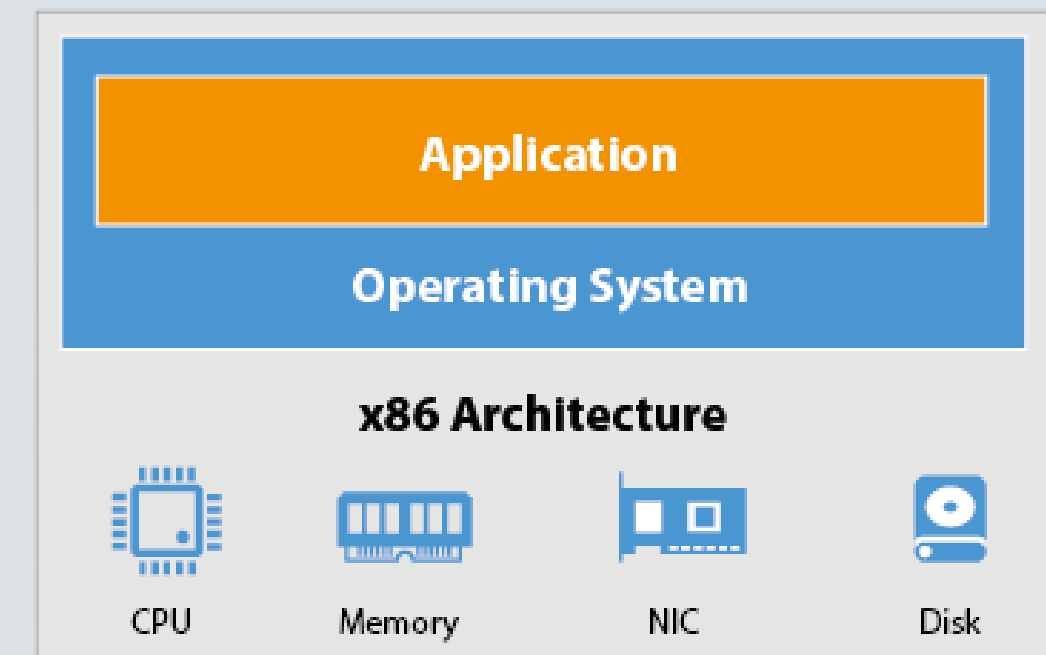
Service web



service database



service application



- L'application est installée directement sur le système d'exploitation du serveur.
- Toutes les dépendances (langages, bibliothèques, services) sont installées manuellement.
- Plusieurs applications peuvent coexister sur le même système, sans réelle isolation.

Le problème avec les serveurs physiques

Gaspillage des ressources

==

Coûts cachés

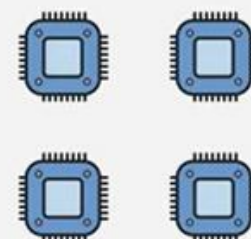
Application

Runtime Environment, Dependancies & Libraries

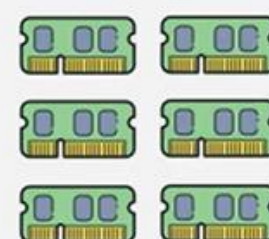
Operating System (Windows, Linux, MacOS)



Motherboard



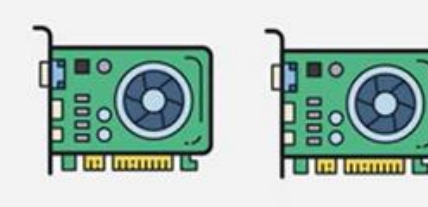
CPUs



Memory

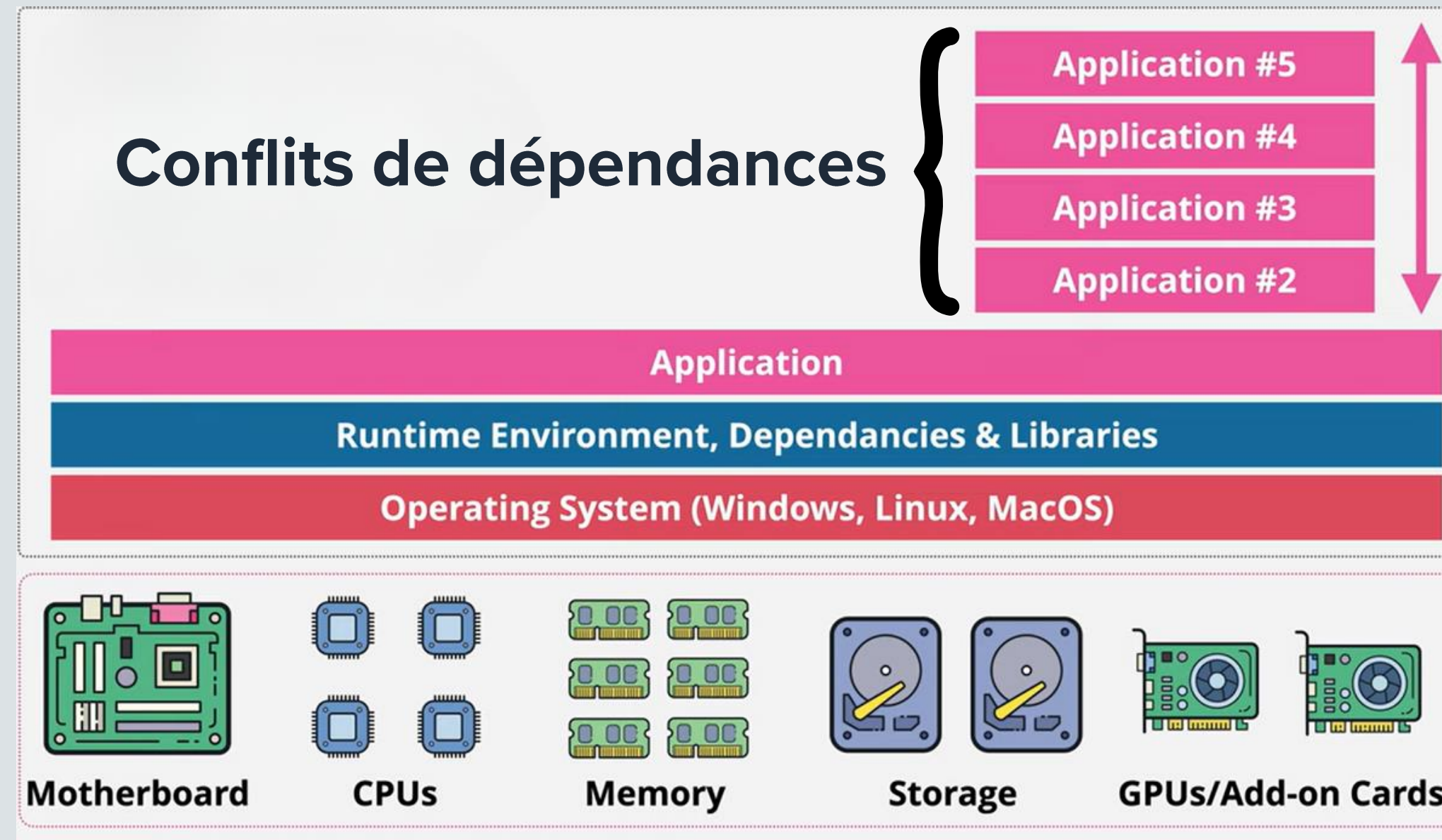


Storage

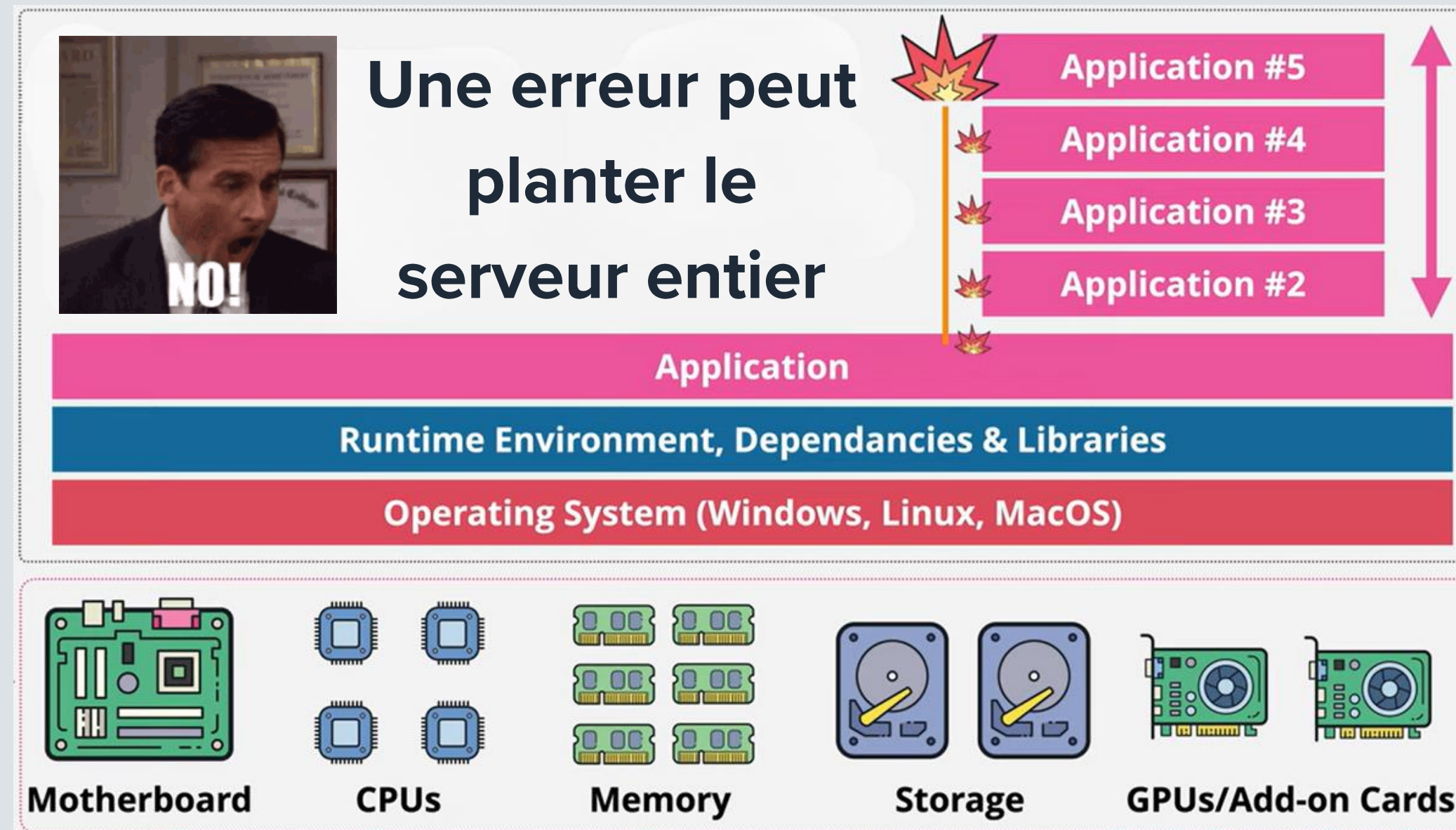


GPUs/Add-on Cards

Le problème avec les serveurs physiques



Le problème avec les serveurs physiques



Le problème avec les serveurs physiques

1. Conflits de dépendances

- Si deux applis ont besoin de versions différentes de Python, Java, etc., cela peut créer des conflits.

2. Isolation inexistante

- Une erreur ou surcharge dans une application peut planter le serveur entier.
- Les processus partagent les mêmes ressources (mémoire, CPU, fichiers système).

3. Difficulté à reproduire

- Recréer le même environnement sur un autre serveur = processus manuel, source d'erreurs.
- "Ça marche sur mon poste" devient un cauchemar en production.

4. Mauvaise scalabilité

- Pour faire tourner une autre instance de l'application, il faut un autre serveur physique.

5. Gaspillage de ressources

- Serveurs souvent sous-utilisés, car on évite d'y mettre trop d'applications à cause du manque d'isolation.

HELLO VMs!!

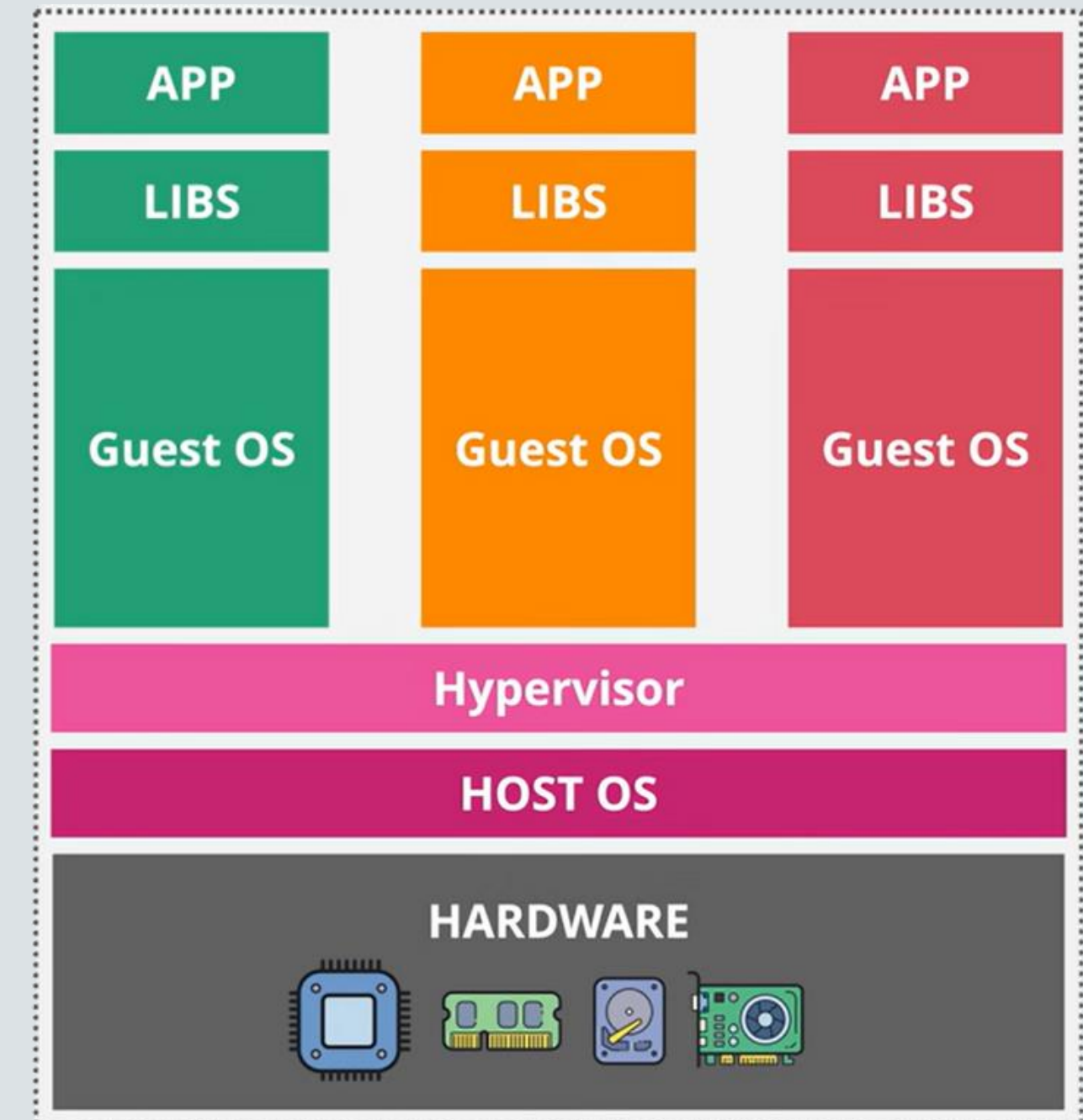
- La communauté informatique disposait enfin d'une technologie permettant d'exécuter de manière sécurisée et fiable plusieurs applications métier sur un seul serveur ... C'est l'idée des VMs!
- Ce fut **une révolution** ! Les équipes IT n'avaient plus besoin d'acquérir un serveur neuf et surdimensionné à chaque fois que l'entreprise demandait une nouvelle application. Bien souvent, les équipes IT utilisaient les serveurs existants, souvent sous-exploités, pour ajouter des applications .

...VMWare en 1999.

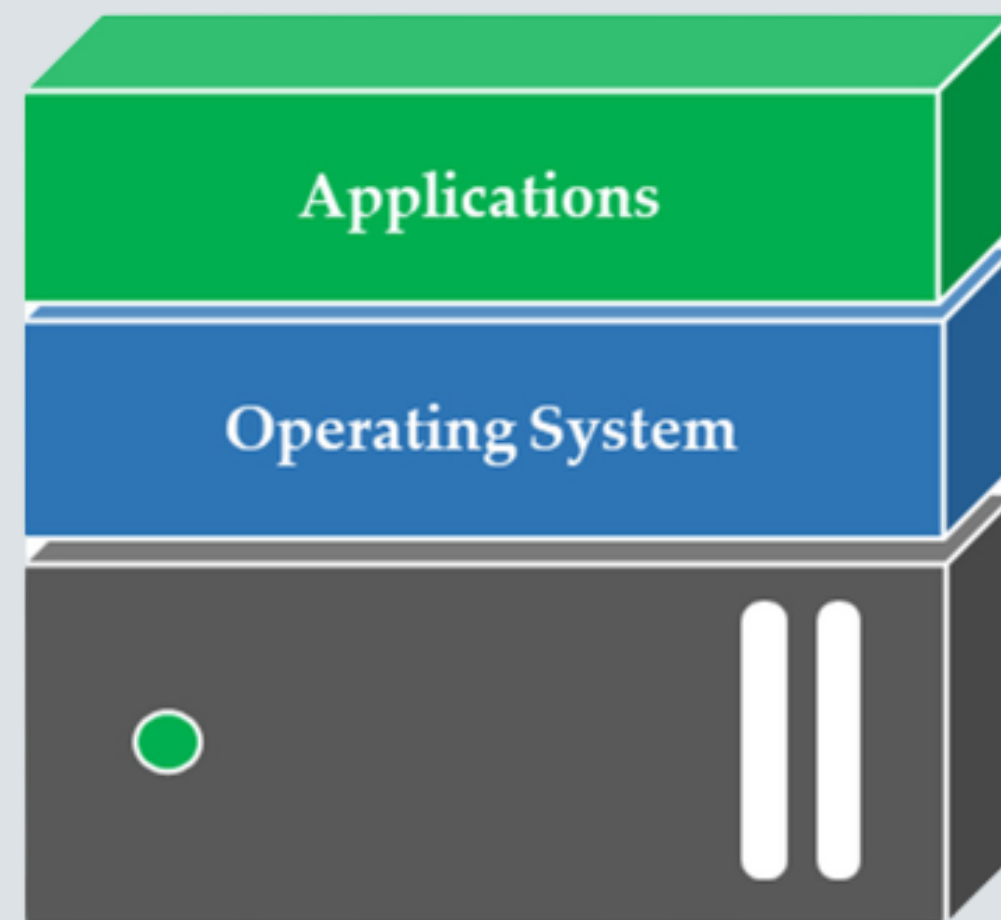


Concept de virtualisation

- on simule des ordinateurs entiers (machines virtuelles) à l'intérieur d'un seul ordinateur physique
 - j'ai compris mais comment elles fonctionnent?
- Sur un serveur physique, on installe un hyperviseur (comme VMware, VirtualBox, Hyper-V, KVM...).
- Cet hyperviseur permet de faire tourner plusieurs VMs indépendantes, chacune avec :
 - Son propre système d'exploitation
 - Ses dépendances
 - Ses applications
- Quand vous créez une VM, vous choisissez :
 - Combien de RAM elle utilisera
 - Combien de CPU (cœurs), etc ...

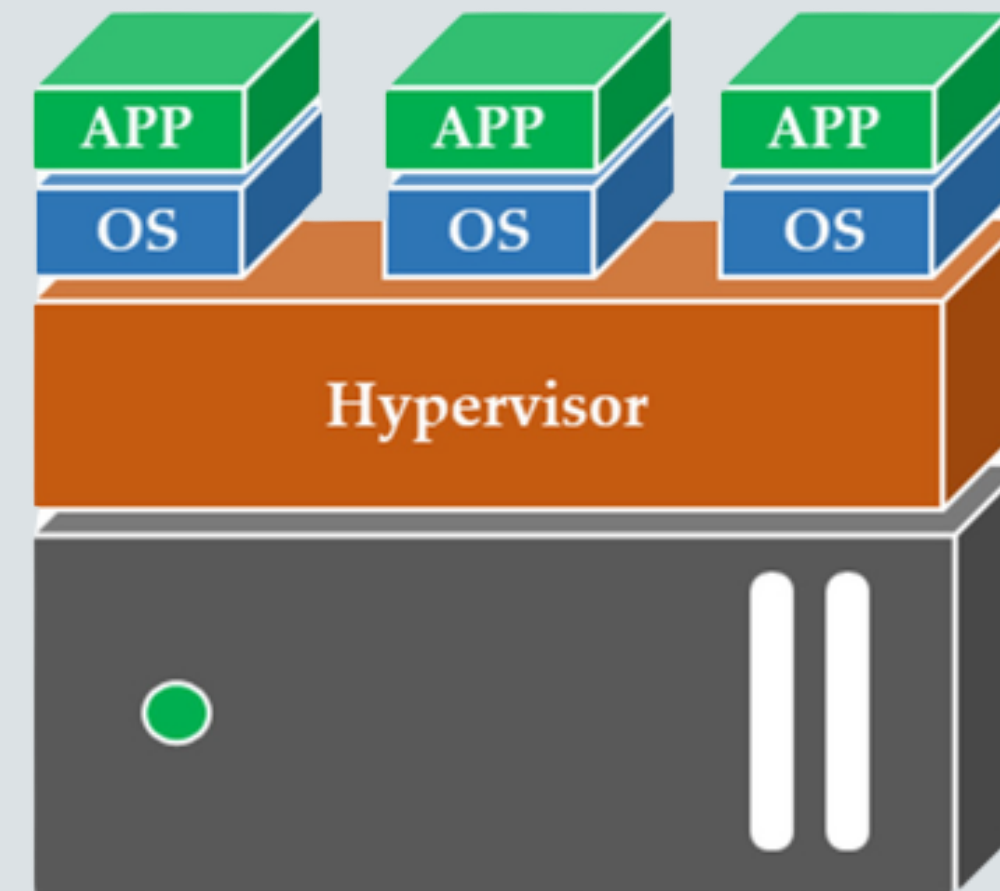


Comparaison



Serveur Physique

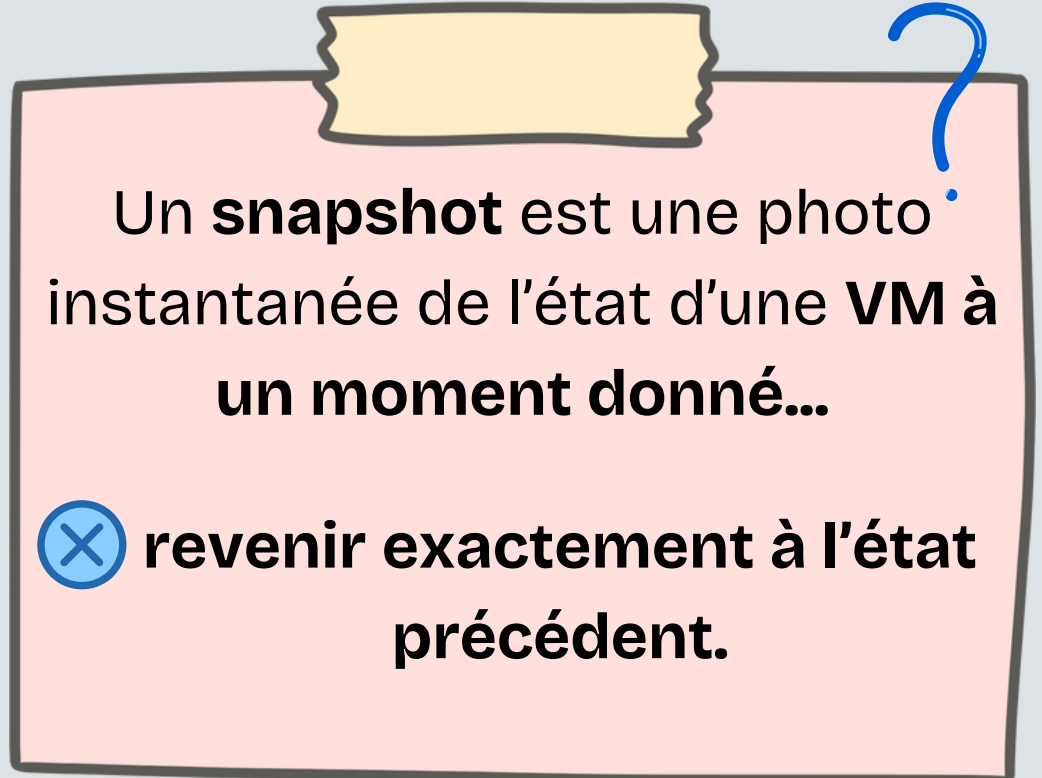
VS



Virtualisation

Comparaison

- les différences clés et Comment les VMs surpassent les limites :
- **Isolation complète:**
 - Chaque VM fonctionne comme un ordinateur indépendant → aucun conflit entre applications.
- **Meilleure utilisation du matériel:**
 - Sur un même serveur physique, on peut exécuter plusieurs VMs ...
- **Reproductibilité et déploiement facile:**
 - Une VM peut être sauvegardée sous forme **d'image (snapshot)** → déploiement rapide de nouvelles machines identiques.



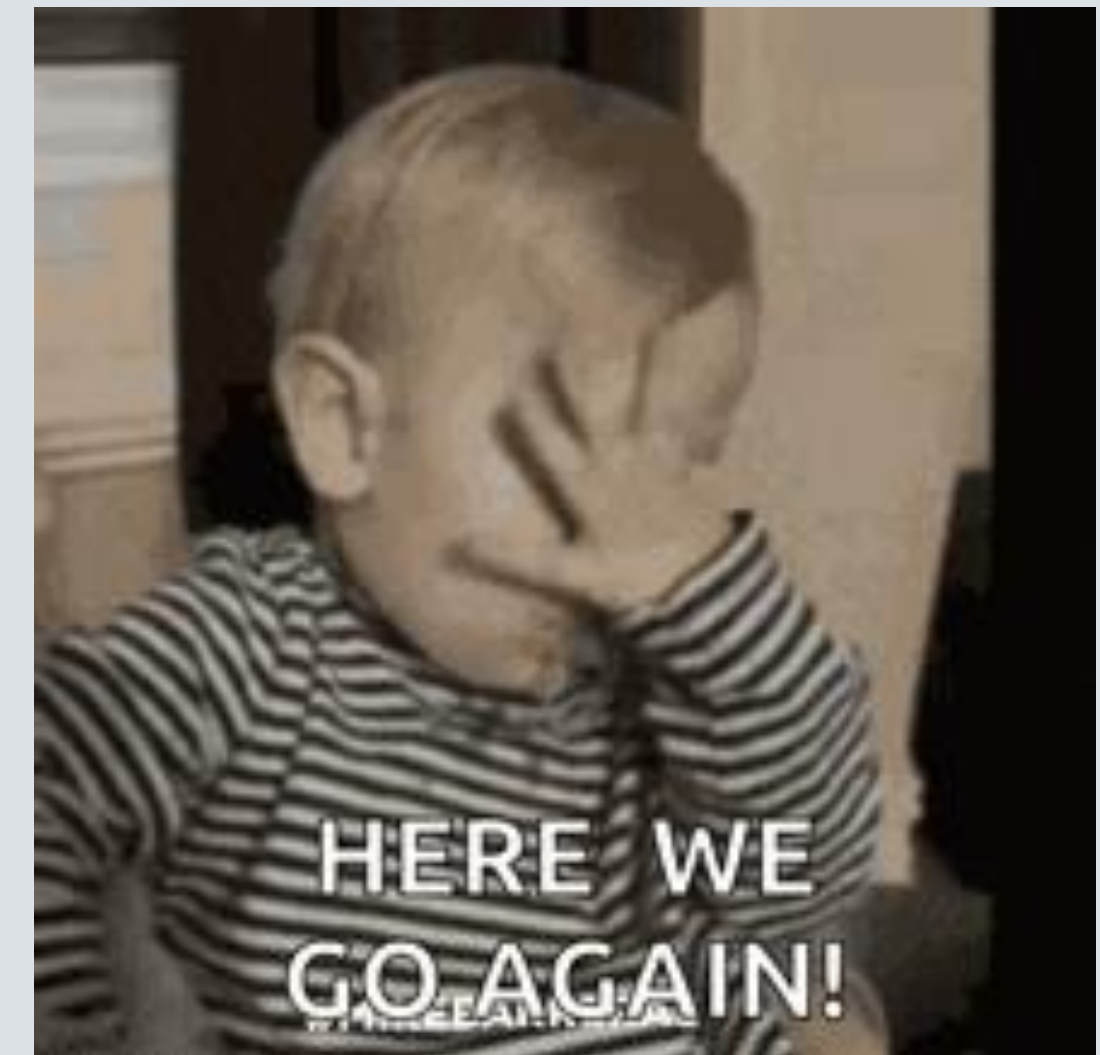
Un **snapshot** est une photo instantanée de l'état d'une **VM** à un moment donné...

⊗ revenir exactement à l'état précédent.

Les Défis accompagnants

.... Cependant, les VMs ont aussi des **inconvénients** :

- **Lourdeur** : Chaque VM nécessite son propre système d'exploitation, consommant beaucoup de ressources.
- **Démarrage Lent** : Lancer une VM prend du temps; d'ailleurs c'est comme une vraie machine: boot du système, démarrage des services...
- **Inefficacité** : Les multiples OS sur une même machine entraînent une grande surcharge.
- **License ...** un autre remarque pour les templates VMs (snapshots).



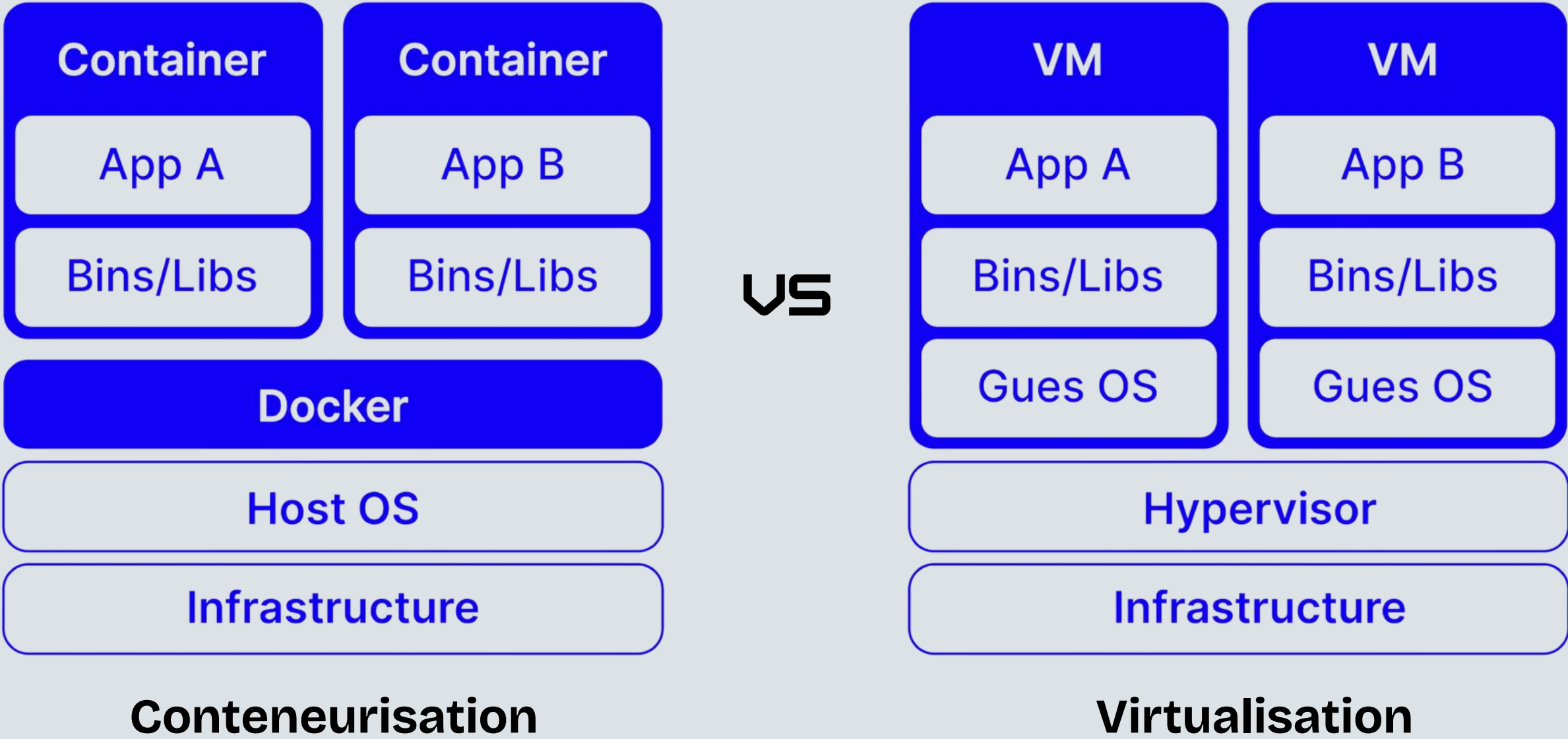
Conteneurisation

- **Les conteneurs Docker apportent une alternative plus légère et plus efficace aux VMs :**
 - **Noyau OS Partagé :** Contrairement aux VMs, les conteneurs partagent le noyau du système d'exploitation hôte, réduisant la redondance et l'utilisation des ressources.
 - **Isolation :** Chaque conteneur est isolé , avec ses propres bibliothèques et dépendances.
 - **Léger et Rapide :** Les conteneurs sont plus petits et démarrent presque instantanément, permettant d'en exécuter des centaines sur une seule machine, contre seulement quelques VMs.

.... en **2013**



Comparaison

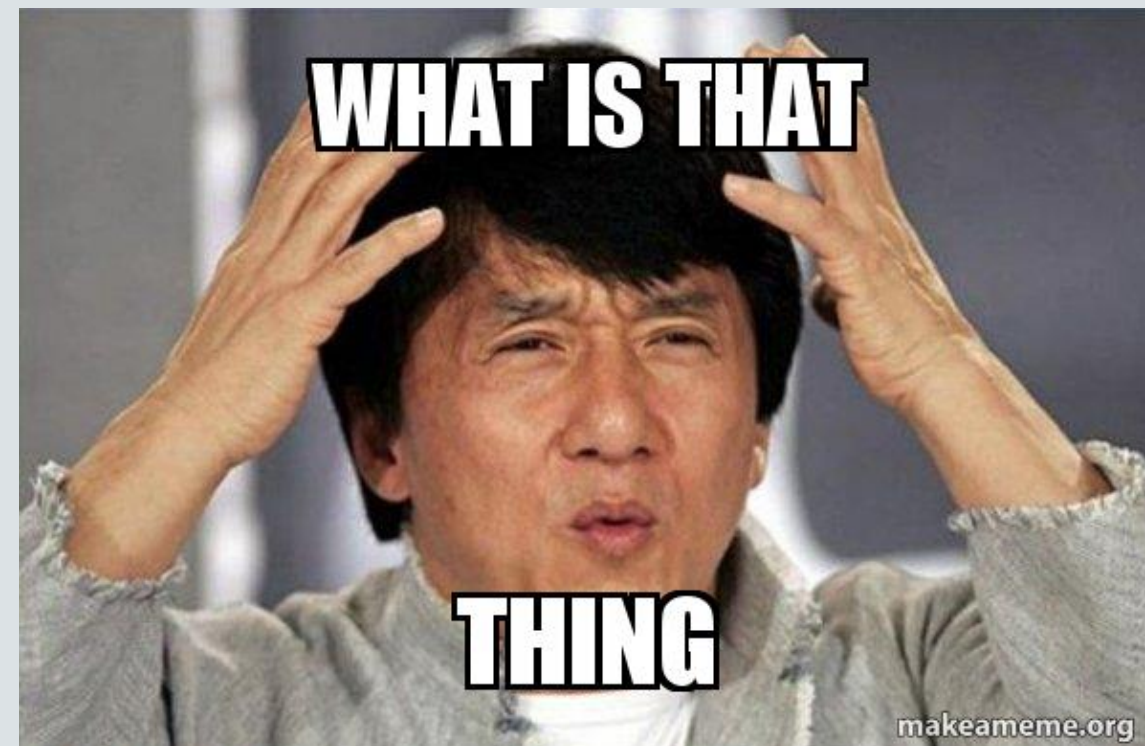


Comparaison

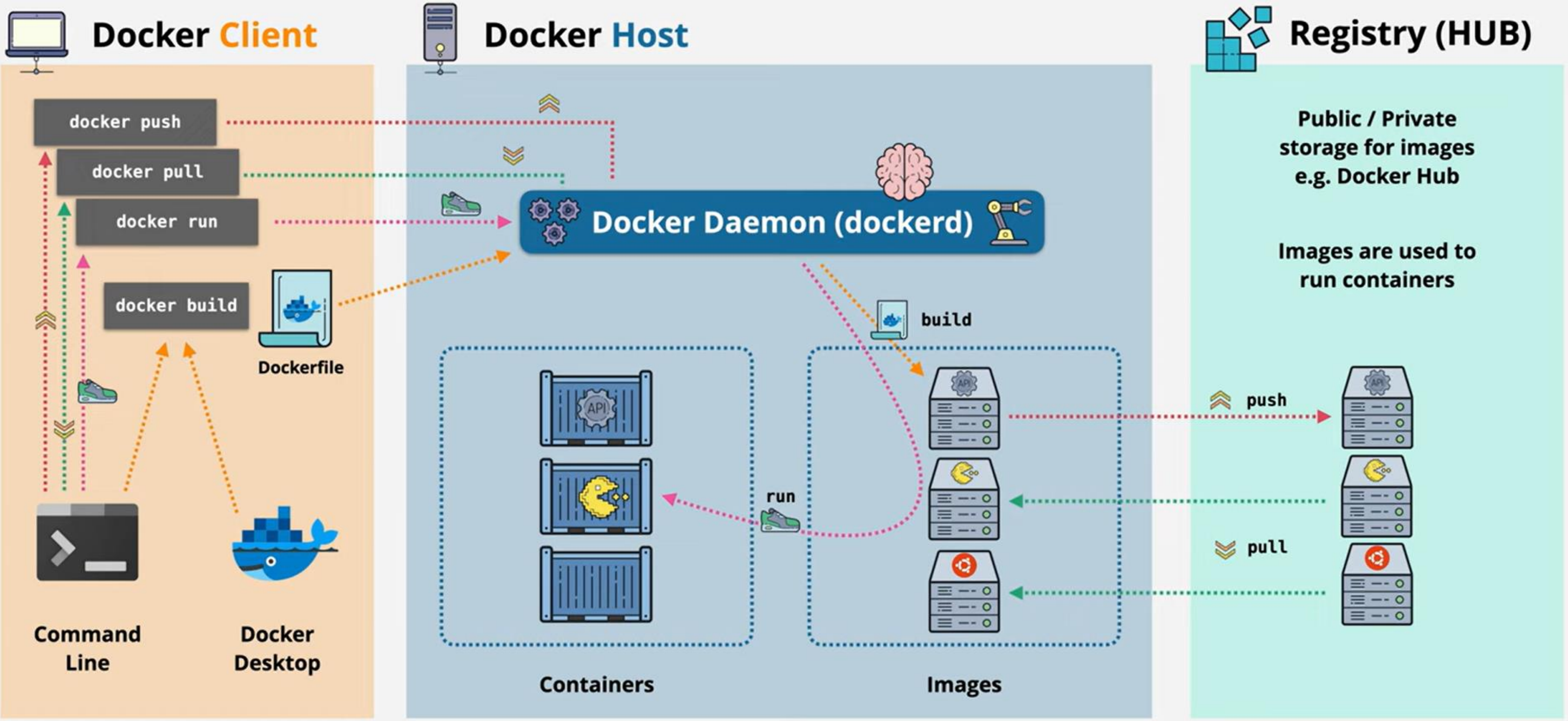
- **Léger et rapide :**
 - Un conteneur démarre en quelques millisecondes.
 - Il consomme peu de ressources.
- **Portabilité totale :**
 - Une application dans un conteneur fonctionne de la même manière partout : sur votre PC, sur un serveur, sur le cloud.
- **Adapté aux microservices :**
 - Chaque service (ex: base de données, API, front-end) peut tourner dans son propre conteneur, facilement déployable et remplaçable.
- **Intégration parfaite avec DevOps :**
 - Docker est devenu un pilier des chaînes CI/CD (intégration et déploiement continus).
 - Il simplifie les tests, la livraison continue et la mise en production

Alors ... 🐳 Qu'est-ce que Docker ?

Docker est une plateforme open-source de conteneurisation. Elle permet d'encapsuler une application avec toutes ses dépendances dans un conteneur léger, isolé et portable, sans avoir besoin d'un système d'exploitation complet comme une VM.



Architecture



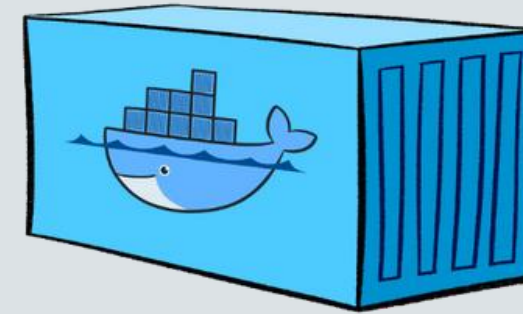
Docker Image



- **C'est quoi?**
 - Une **image Docker** est un modèle figé, **en lecture seule**, qui contient tout **le nécessaire** pour exécuter une application **dans un conteneur**. (stopped container, similaire à un class...)
 - C'est une sorte de **blueprint** ou de "**photo complète**" d'un environnement d'exécution (comme un VM template).
- **Il est composé de:**
 - Le système de base (ex: Debian, Alpine...)
 - Les dépendances (bibliothèques, runtimes...)
 - Le code source de l'app
 - Les fichiers de config
 - Le point d'entrée (la commande qui démarre l'app)

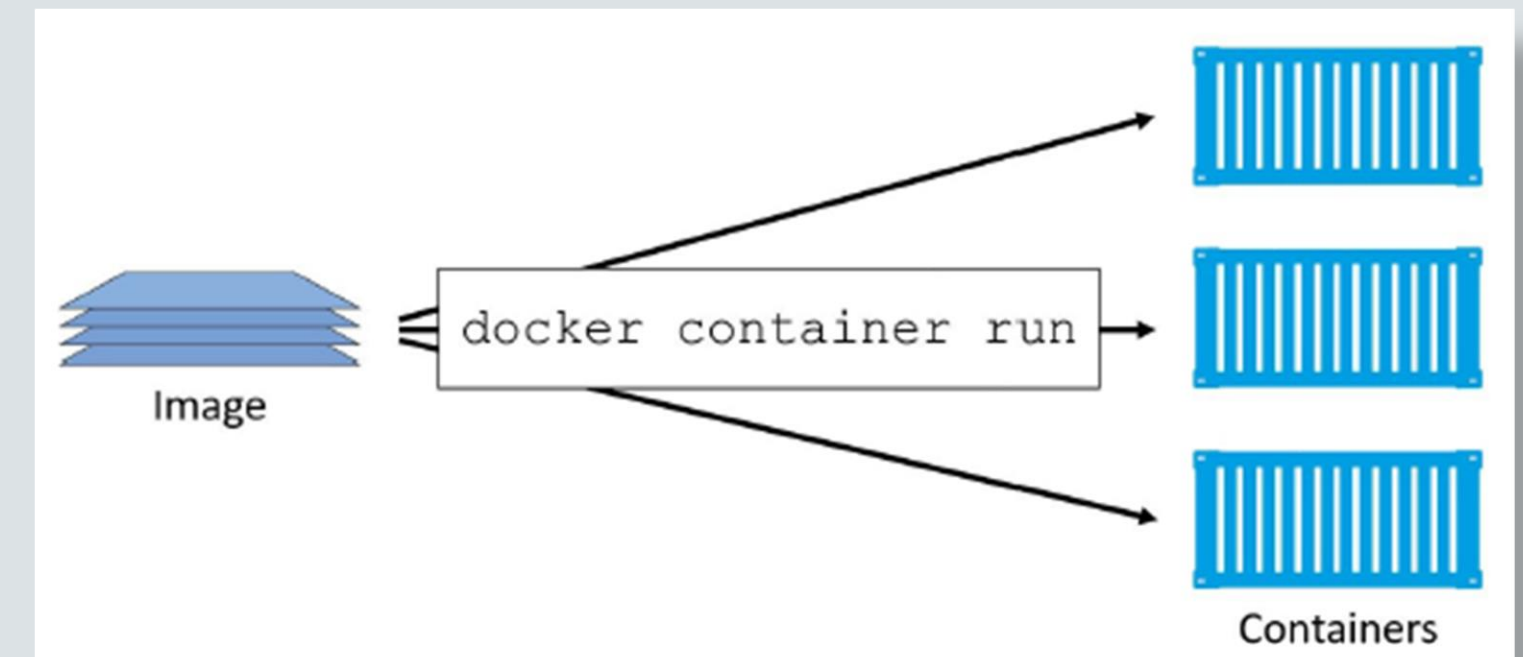


Docker Container



- **Un conteneur Docker est :**

- Une **boîte isolée, Légère et exécutable** qui contient :
 - le code de votre application,
 - ses dépendances (bibliothèques, exécutables, fichiers...),
 - sa configuration (variables d'environnement, ports...).



- **Instances actives d'images**, s'exécutant de manière isolée tout en **partageant le noyau de l'OS**.

« Un conteneur est comme une machine virtuelle légère et instantanée ! »

Dockerfile



- À quoi ça sert ?
 - C'est le mode d'emploi pour construire une image Docker contenant votre application
- Image Docker (Construction)
 - Commande : **docker build -t mon_image .**
 - Résultat :
 - Package complet de l'application
 - Environnement configuré
 - Immuable (ne change plus après construction)
- Conteneur (Exécution)
 - Commande : **docker run -p 3000:3000 mon_image**
 - Caractéristiques :
 - Instance isolée de l'image
 - Modifiable (avec volumes)
 - Portable sur tout système Docker

```
# Image de base
FROM node:latest

# Définit le dossier de travail dans l'image
WORKDIR /app

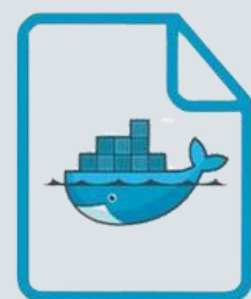
# Copie des fichiers locaux dans l'image
COPY package.json .

# Exécute une commande lors de la construction
RUN npm install

# Copie du code source
COPY . .

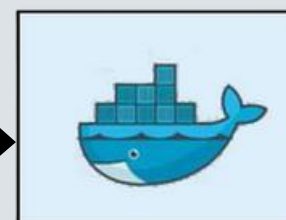
# Documentation du port
EXPOSE 3000

# Commande exécutée à l'exécution du conteneur
CMD ["node", "app.js"]
```



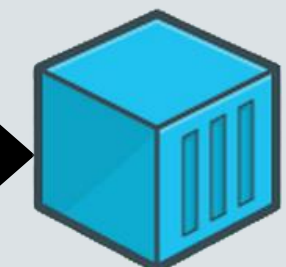
Dockerfile

(**docker build -t mon_image .**)



Docker
Image

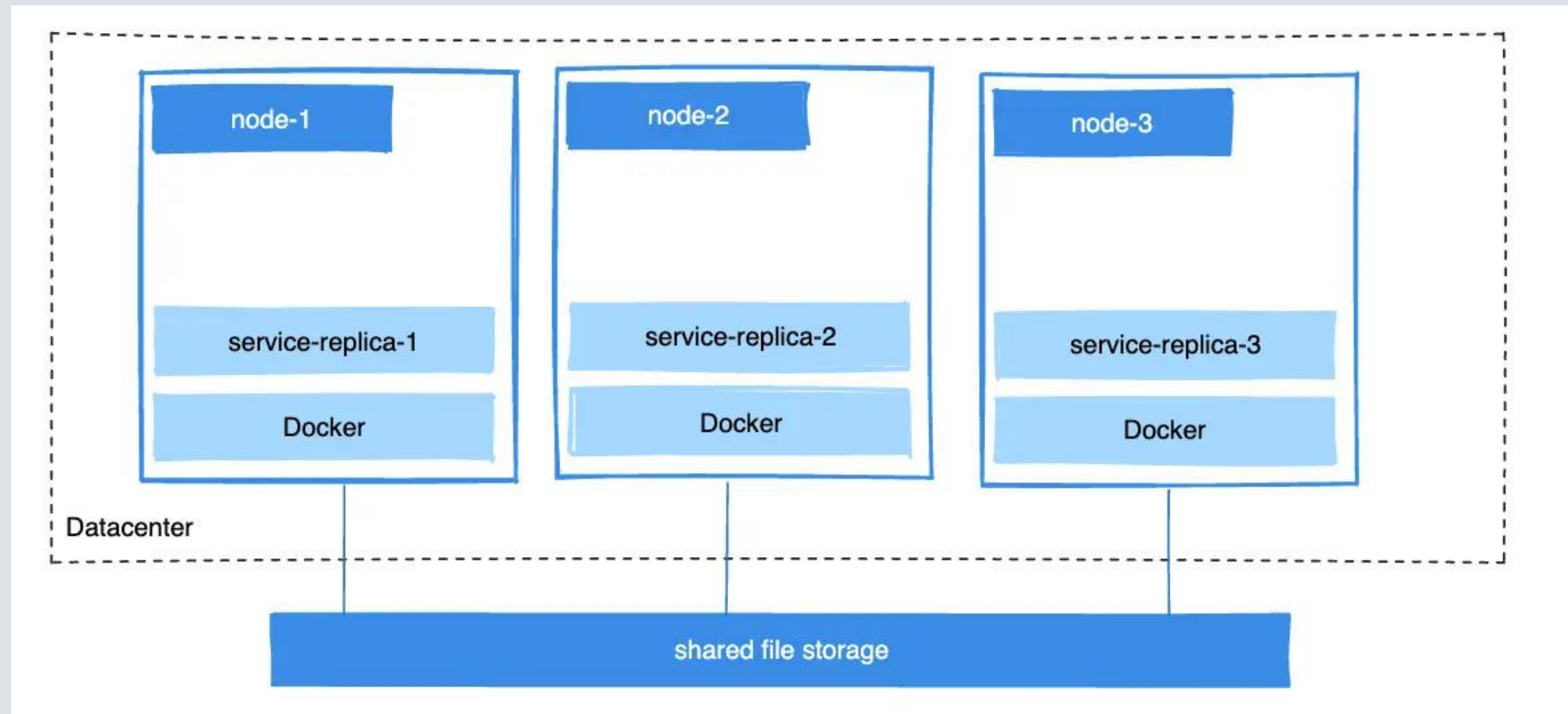
(**docker run -p 3000:3000 mon_image**)



Docker
Container

Volumes

- Permet de sauvegarder des données même après la suppression du conteneur
- Idéal pour les bases de données ou le stockage d'état



Création du Volume Docker

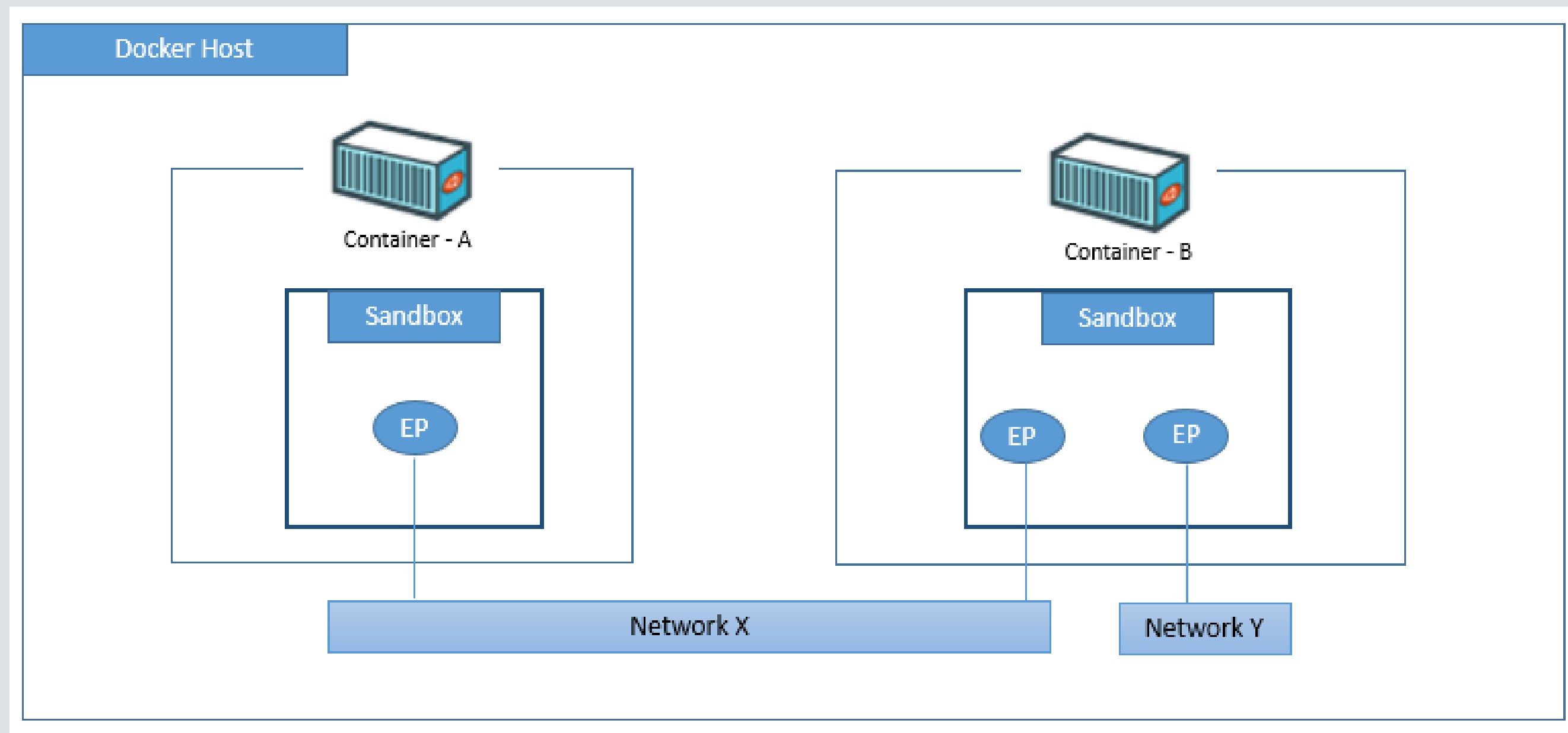
- **Action :** Crée un espace de stockage nommé mongo_data géré par Docker
- **À retenir :**
 - C'est comme une clé USB virtuelle pour vos conteneurs
 - Les données sont stockées en dehors du conteneur
 - Survit même si le conteneur est supprimé



```
>> docker volume create mongo_data  
>> docker run -d --name mongo -v mongo_data:/data/db mongo
```

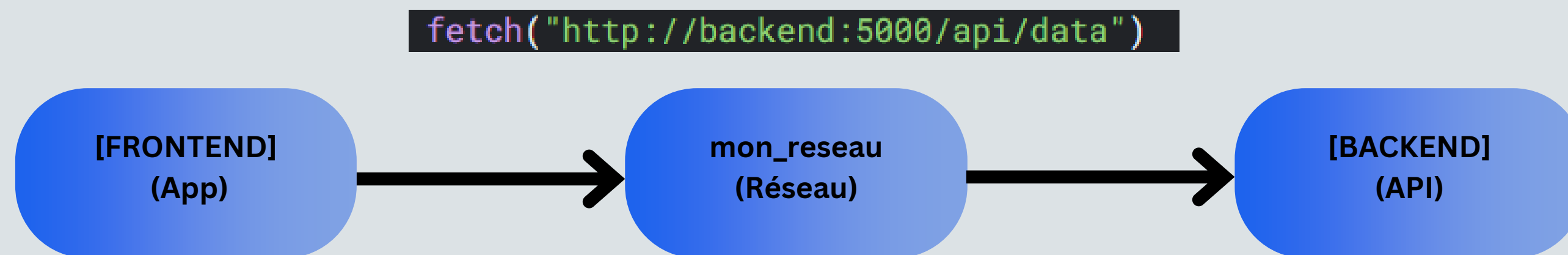
Réseaux (Network)

- Permettent aux conteneurs de communiquer entre eux sur un réseau privé
- Chaque conteneur peut être joint via son nom de service grâce au DNS interne Docker



Création du Réseau Docker

- **Action :** Crée un réseau virtuel nommé **mon_reseau** spécialement pour vos conteneurs.
- **À retenir :**
 - Les conteneurs connectés à ce réseau peuvent se "parler" entre eux
 - Ils se trouvent automatiquement par leur nom (comme un annuaire téléphonique interne)
 - C'est comme créer un groupe privé de discussion pour vos applications



```
fetch("http://backend:5000/api/data")
```

```
>> docker network create mon_reseau
>> docker run -d --name backend --network mon_reseau mon_api
>> docker run -d --name frontend --network mon_reseau mon_app
```

Docker Compose

Docker Compose est un outil qui permet de :

- Définir** et **lancer** plusieurs conteneurs Docker ensemble
- À partir d'un **seul fichier YAML** (docker-compose.yml)
- Sans avoir à taper des commandes complexes pour chaque conteneur

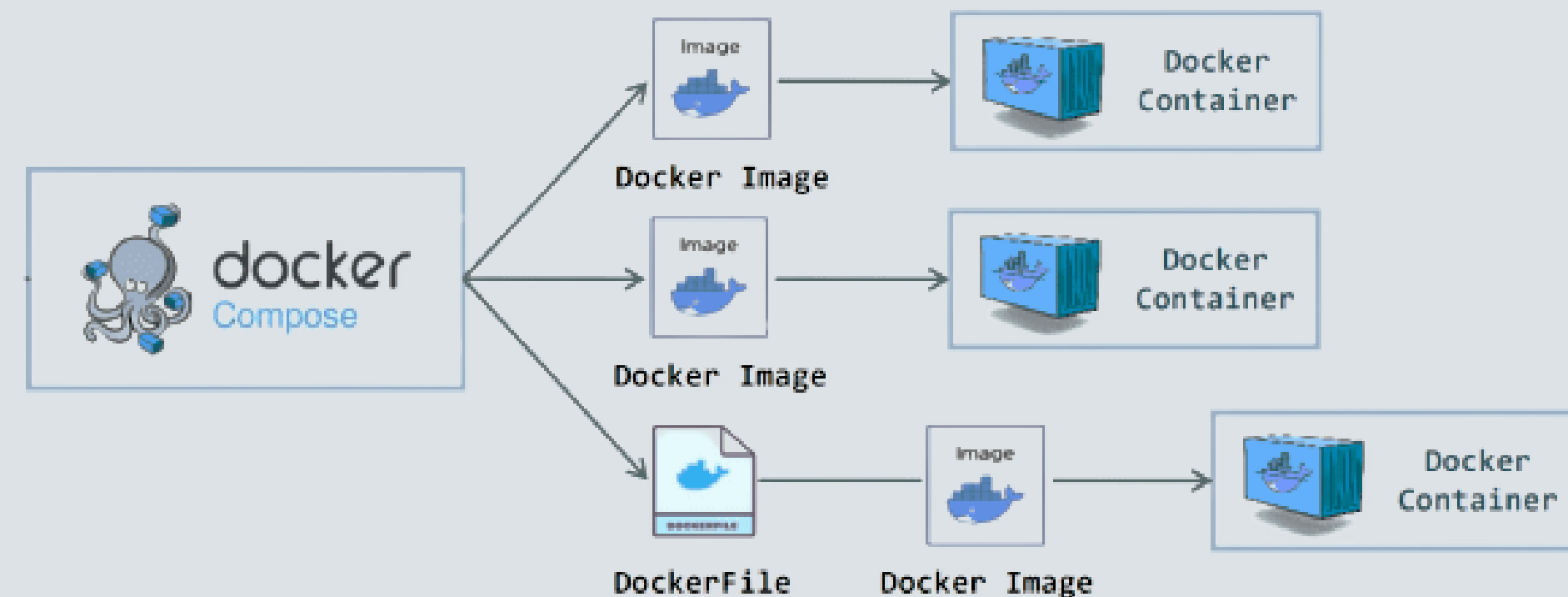
Analogie :

Imaginez une recette de cuisine qui décrit :

- Tous les ingrédients (vos services : base de données, backend, frontend)
- Comment ils s'assemblent (connexions entre eux)
- Les instructions de cuisson (ports, volumes, etc.)

À quoi ça sert ?

- Développer des applications multi-conteneurs (ex : app + base de données)
- Configurer facilement les réseaux et volumes partagés
- Démarrer/arrêter toute l'application d'un seul coup



Docker Compose

[Structure du Projet]

- **[Dockerfile Backend]** → Construit l'image backend (Node.js, port 5000)
- **[Dockerfile Frontend]** → Construit l'image frontend (React + Nginx, port 80)
- **[docker-compose.yml]** → Orchestre les 3 services:
 - **MongoDB (image officielle)**
 - Port 27017
 - Volume mongo_data
 - **Backend (build: ./backend)**
 - Port 5000
 - Dépend de MongoDB
 - **Frontend (build: ./frontend)**
 - Port 3000:80
 - Dépend du Backend

```

docker-compose.yml > YAML > version
1  version: "3.8"
2  services:
3    mongo:
4      image: mongo
5      volumes:
6        - mongo_data:/data/db
7      ports:
8        - "27017:27017"
9
10   backend:
11     build: ./backend
12     ports:
13       - "5000:5000"
14     depends_on:
15       - mongo
16
17   frontend:
18     build: ./frontend
19     ports:
20       - "3000:80"
21     depends_on:
22       - backend
23
24   volumes:
25     mongo_data:
    
```

Flux d'exécution :

1. **docker-compose build** → Construit les images depuis les Dockerfiles
2. **docker-compose up** → Lance les conteneurs dans l'ordre :
 - **MongoDB d'abord** (car backend/frontend en dépendent)
 - **Puis backend**
 - **Enfin frontend**

Connexions automatiques :

- Frontend peut joindre backend via **<http://backend:5000>**
- Backend peut joindre MongoDB via **<mongodb://mongo:27017>**

Conclusion

- Docker facilite la portabilité, efficacité et rapidité de déploiement des applications
- Conteneurs = isolation + cohérence entre développement et production
- Évite les conflits de dépendances
- Simplifie et rationalise le déploiement
- Un outil incontournable pour les développeurs modernes

Références

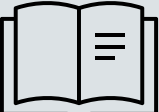
 <https://www.oracle.com/ma/cloud/cloud-native/container-registry/what-is-docker/>

 <https://docs.docker.com/get-started/>

 wikipedia.org/wiki/Machine_virtuelle

 <https://github.com/acantril/docker-fundamentals>

 <https://github.com/ahmedsami76/AraBigData/blob/main/Docker.ipynb>

 Docker Deep Dive: Zero to Docker in a single book Kindle Edition

MERCY POUR
VOTRE
ATTENTION